

Exercise 1 — Sending a Simple GET Request

Description: Use `HttpClient` (Java 11+) to send a GET request to a public endpoint, for example `https://api.github.com/`. Retrieve and print the status code, headers, and body of the response.

Requirements: - Initialize `HttpClient` with `HttpClient.newHttpClient()`. - Build an `HttpRequest` with `HttpRequest.newBuilder()` and the corresponding URL. - Call `client.send()` or `client.sendAsync()` to obtain an `HttpResponse`. - Print the response code (`response.statusCode()`), headers (`response.headers()`), and body (`response.body()`).

Exercise 2 — GET Request with Query Parameters

Description: Send a GET request to a public API that takes a query parameter, for example `https://api.agify.io?name=michael` (an API that predicts age based on a name). Extract data from the JSON response and print the name and predicted age.

Requirements: - Use `HttpRequest` to send a GET request with query parameters. - Use `ObjectMapper` (from Jackson) or a similar library to parse the JSON. - Print information such as: Name: michael, Age: xx.

Exercise 3 — POST Request with a JSON Payload

Description: Create a POST request sent to a mock server (e.g. `https://httpbin.org/post`) with a simple JSON body (e.g. `{"username": "john", "password": "1234"}`). Parse the server's response and print its content.

Requirements: - Create a POST `HttpRequest`, setting the header `Content-Type: application/json`. - The body is the JSON string. - Send the request, receive the response, parse the JSON response, and print it.

Exercise 4 — PUT and DELETE Requests

Description: Assume you have a mock REST API at `https://jsonplaceholder.typicode.com/posts`. - Send a PUT request to `https://jsonplaceholder.typicode.com/posts/1` to update a mock post (e.g. its title, body). - Send a DELETE request to `https://jsonplaceholder.typicode.com/posts/1` to delete the post. - Check the status code of each response.

Requirements: - Use a PUT request with a new JSON payload. - Check the response code (200, 204, etc.). - Send a DELETE request and check the response code.

Exercise 5 — Error and Exception Handling

Description: Send a GET request to a URL that does not exist or that causes an error (e.g. `https://api.github.com/thispagedoesnotexist`). Catch and print the 404 error from the response. Simulate a connection timeout and handle the `HttpTimeoutException`.

Requirements: - Configure `HttpRequest` with `timeout ( )`. - Try sending a request to an invalid URL to see how to handle 4xx/5xx status codes. - Catch exceptions and print an appropriate error message.